
Gestión de Estado

Sesiones y Carrito de Compras

Materia: Aplicaciones Web

Plataformas: PHP 8 | ASP.NET Core 8 | Spring Boot 3

Estudiante: Carla

Fecha: 10 de julio de 2026

Git: <https://github.com/carla22072004/TAREA-FORMATIVA-APP.git>

Resumen

El presente documento describe la implementación comparativa de un sistema de carrito de compras basado en sesiones de servidor, utilizando tres tecnologías web: **PHP 8**, **ASP.NET Core 8** y **Java / Spring Boot 3**. Se abordan la configuración de cookies seguras (HttpOnly, SameSite, Secure), la persistencia de objetos en sesión y el patrón **PRG** (Post/Redirect/Get).

Índice general

1. Marco Teórico	2
1.1. Gestión de Estado en Aplicaciones Web	2
1.1.1. Sesiones de Servidor	2
1.1.2. Seguridad en Cookies de Sesión	2
1.1.3. Patrón PRG (Post / Redirect / Get)	3
2. Plataforma 1: PHP 8	4
2.1. Estructura de Archivos	4
2.2. Implementación	4
2.2.1. configuracion.php — Seguridad de la Sesión	4
2.2.2. agregar.php — Patrón PRG	5
2.2.3. eliminar.php	5
2.3. Capturas de Pantalla	6
3. Plataforma 2: ASP.NET Core 8	7
3.1. Estructura del Proyecto	7
3.2. Implementación	7
3.2.1. Program.cs — Registro de Servicios y Middleware	7
3.2.2. Models/Producto.cs	8
3.2.3. Controllers/CarritoController.cs	8
3.3. Capturas de Pantalla	10
4. Plataforma 3: Java / Spring Boot 3	11
4.1. Estructura del Proyecto Maven	11
4.2. Implementación	11
4.2.1. pom.xml — Dependencias	11
4.2.2. application.yml — Cookie Segura	12
4.2.3. model/Producto.java	12
4.2.4. controller/CarritoController.java	12
4.3. Capturas de Pantalla	14
5. Análisis Comparativo	15
5.1. Comparación entre Plataformas	15
5.2. Resumen de Flags de Seguridad	15
6. Conclusiones	16

Capítulo 1

Marco Teórico

1.1 Gestión de Estado en Aplicaciones Web

HTTP es un protocolo **sin estado** (*stateless*): cada petición es independiente y el servidor no recuerda peticiones anteriores. Para mantener información entre peticiones (carrito, usuario autenticado, etc.) se utilizan mecanismos de **gestión de estado**.

1.1.1 Sesiones de Servidor

Una **sesión** es un almacén de datos que el servidor asocia a un cliente mediante un identificador único llamado **Session ID**, que se transmite en una *cookie*. El flujo básico es:

1. El cliente realiza su primera petición al servidor.
2. El servidor genera un Session ID único y lo almacena.
3. El ID se envía al cliente en una cookie de respuesta.
4. En peticiones siguientes, el cliente reenvía la cookie.
5. El servidor recupera los datos de sesión asociados a ese ID.

1.1.2 Seguridad en Cookies de Sesión

Para proteger el Session ID se deben configurar las siguientes banderas:

Cuadro 1.1: Flags de seguridad para cookies de sesión

Flag	Descripción
HttpOnly	Impide el acceso al cookie desde JavaScript (previene XSS).
Secure	La cookie solo se transmite sobre HTTPS (previene sniffing).
SameSite	Controla el envío de cookies en peticiones cruzadas (previene CSRF).
Strict	Valor de SameSite: cookie solo se envía en peticiones del mismo sitio.

Capítulo 2

Plataforma 1: PHP 8

PHP 8 — Carrito de Compras con \$_SESSION

PHP gestiona sesiones con variables superglobales como \$_SESSION. La configuración de seguridad debe establecerse con ini_set() **antes** de llamar a session_start().

2.1 Estructura de Archivos

Cuadro 2.1: Archivos del proyecto PHP

Archivo	Función
configuracion.php	Configuración de seguridad y arranque de sesión
index.php	Vista principal: catálogo + carrito
agregar.php	Procesa POST y agrega producto a \$_SESSION
eliminar.php	Elimina un elemento por índice con array_splice
limpiar.php	Vacía el carrito de la sesión

2.2 Implementación

2.2.1 configuracion.php — Seguridad de la Sesión

```
1 <? php
2 // Configurar ANTES de session_start()
3 ini_set('session.cookie_httponly', '1'); // Flag HttpOnly
4 ini_set('session.cookie_secure', '0'); // 1 en
   produccion (HTTPS)
5 ini_set('session.cookie_samesite', 'Strict'); // Anti-CSRF
6 ini_set('session.use_strict_mode', '1'); // IDs
   regenerados
7 ini_set('session.gc_maxlifetime', '1800'); // 30 minutos
8
9 session_start();
10
11 // Regenerar ID al iniciar sesion (previene Session Fixation)
```

```
12 if (!isset($_SESSION['iniciada'])) {
13     session_regenerate_id(true);
14     $_SESSION['iniciada'] = true;
15 }
```

Listing 2.1: configuracion.php: Configuración segura antes de session_start()

2.2.2 agregar.php — Patrón PRG

```
1 <? php
2 require 'configuracion.php';
3
4 // Precios validos - NUNCA confiar en datos del cliente
5 $precios = [
6     'Laptop' => 450,
7     'Mouse' => 25,
8     'Teclado' => 45,
9     'Monitor' => 180,
10 ];
11
12 $nombre = filter_input(INPUT_POST, 'nombre',
13     FILTER_SANITIZE_SPECIAL_CHARS);
14
15 if ($nombre !== null && array_key_exists($nombre, $precios)) {
16     if (!isset($_SESSION['carrito'])) {
17         $_SESSION['carrito'] = [];
18     }
19     $_SESSION['carrito'][] = [
20         'nombre' => $nombre,
21         'precio' => $precios[$nombre],
22     ];
23 }
24
25 // Patron Post/Redirect/Get
26 header('Location: index.php');
27 exit;
```

Listing 2.2: agregar.php: Validación del lado del servidor y PRG

2.2.3 eliminar.php

```
1 <? php
2 require 'configuracion.php';
3
4 $indice = filter_input(INPUT_POST, 'indice',
5     FILTER_VALIDATE_INT);
6
7 if ($indice !== false && $indice !== null
8     && isset($_SESSION['carrito'][$indice])) {
9     array_splice($_SESSION['carrito'], $indice, 1);
10 }
```

```
9 }
10
11 header('Location: index.php');
12 exit;
```

Listing 2.3: eliminar.php: Eliminación segura por índice

Capítulo 3

Plataforma 2: ASP.NET Core 8

ASP.NET Core 8 — Middleware ISession

ASP.NET Core gestiona sesiones mediante el middleware ISession, que requiere un proveedor de caché distribuido. Los objetos complejos deben serializarse a JSON, ya que ISession solo acepta byte[] y string.

3.1 Estructura del Proyecto

Cuadro 3.1: Archivos del proyecto ASP.NET Core

Archivo	Función
Program.cs	Pipeline de la aplicación, servicios de sesión
Models/Producto.cs	Modelo de datos (record inmutable)
Controllors/CarritoController.cs	Lógica de negocio del carrito
Views/Carrito/Index.cshtml	Vista Razor

3.2 Implementación

3.2.1 Program.cs — Registro de Servicios y Middleware

```
1 var builder = WebApplication.CreateBuilder(args);
2 builder.Services.AddControllersWithViews();
3
4 // 1. Almacenamiento en memoria para sesiones
5 builder.Services.AddDistributedMemoryCache();
6
7 // 2. Configurar sesion con cookie segura
8 builder.Services.AddSession(options =>
9 {
10     options.IdleTimeout = TimeSpan.FromMinutes(30);
11     options.Cookie.HttpOnly = true; // flag
12     options.Cookie.SecurePolicy = CookieSecurePolicy.Always; // flag
13     options.Cookie.IsEssential = true; // flag
14 });
```

```

14     options.Cookie.SameSite = SameSiteMode.Strict;    // flag
        SameSite
15     options.Cookie.IsEssential = true;
16 });
17
18 var app = builder.Build();
19
20 app.UseHttpsRedirection();
21 app.UseRouting();
22
23 // 3. Habilitar middleware de sesion (ANTES de controladores)
24 app.UseSession();
25
26 app.UseAuthorization();
27 app.MapStaticAssets();
28 app.MapControllerRoute (
29     name: "default",
30     pattern: "{controller=Carrito}/{action=Index}/{id?}"
31     .WithStaticAssets ();
32
33 app.Run ();

```

Listing 3.1: Program.cs: Configuración del pipeline con sesiones seguras

3.2.2 Models/Producto.cs

```

1 namespace CarritoASP.Models;
2
3 // Record: immutable, igualdad por valor, conciso
4 public record Producto(string Nombre, decimal Precio);

```

Listing 3.2: Producto.cs: Record immutable con igualdad por valor

3.2.3 Controllers/CarritoController.cs

```

1 using CarritoASP.Models;
2 using Microsoft.AspNetCore.Mvc;
3 using System.Text.Json;
4
5 public class CarritoController : Controller
6 {
7     private const string CLAVE_CARRITO = "carrito";
8
9     private static readonly Dictionary<string, decimal> Precios
10         = new()
11     {
12         ["Laptop"] = 450, ["Mouse"] = 25,
13         ["Teclado"] = 45, ["Monitor"] = 180,
14     };

```

```

15     private List<Producto> ObtenerCarrito ()
16     {
17         var json = HttpContext.Session.GetString (CLAVE_CARRITO)
18         ;
19         return json is null
20             ? new List<Producto>()
21             : JsonSerializer.Deserialize<List<Producto>>(json)
22             !;
23     }
24
25     private void GuardarCarrito (List<Producto> carrito) =>
26         HttpContext.Session.SetString (CLAVE_CARRITO ,
27         JsonSerializer.Serialize (carrito));
28
29     public IActionResult Index ()
30     {
31         ViewBag.Carrito = ObtenerCarrito ();
32         ViewBag.Catalogo = Precios.Keys.ToList ();
33         ViewBag.SessionId = HttpContext.Session.Id;
34         return View ();
35     }
36
37     [HttpPost]
38     public IActionResult Agregar (string nombre)
39     {
40         if (Precios.TryGetValue (nombre, out var precio))
41         {
42             var carrito = ObtenerCarrito ();
43             carrito.Add (new Producto (nombre, precio));
44             GuardarCarrito (carrito);
45         }
46         return RedirectToAction (nameof (Index)); // PRG
47     }
48
49     [HttpPost]
50     public IActionResult Eliminar (int indice)
51     {
52         var carrito = ObtenerCarrito ();
53         if (indice >= 0 && indice < carrito.Count)
54             carrito.RemoveAt (indice);
55         GuardarCarrito (carrito);
56         return RedirectToAction (nameof (Index));
57     }
58
59     [HttpPost]
60     public IActionResult Limpiar ()
61     {
62         HttpContext.Session.Remove (CLAVE_CARRITO);
63         return RedirectToAction (nameof (Index));
64     }

```


Listing 3.3: CarritoController.cs: Acciones del carrito con serialización JSON

Capítulo 4

Plataforma 3: Java / Spring Boot 3

Spring Boot 3 — HttpSession + Thymeleaf

Spring Boot gestiona sesiones mediante la interfaz HttpSession inyectada directamente en los métodos del controlador. Los objetos almacenados en sesión **deben implementar Serializable**. Las vistas se construyen con **Thymeleaf**.

4.1 Estructura del Proyecto Maven

Cuadro 4.1: Archivos principales del proyecto Spring Boot

Archivo	Función
pom.xml	Dependencias Maven (web + thymeleaf)
application.yml	Configuración del servidor y cookie
CarritoApplication.java	Clase principal con @SpringBootApplication
model/Producto.java	Record serializable
controller/CarritoController.java	Controlador MVC
templates/carrito/index.html	Plantilla Thymeleaf

4.2 Implementación

4.2.1 pom.xml — Dependencias

```

1 <dependencies>
2   <!-- Spring MVC -->
3   <dependency>
4     <groupId>org.springframework.boot</groupId>
5     <artifactId>spring-boot-starter-web</artifactId>
6   </dependency>
7   <!-- Motor de plantillas Thymeleaf -->
   <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-thymeleaf</artifactId>
   </dependency>

```

```
12 </ dependencies >
```

Listing 4.1: pom.xml: Dependencias Spring Web y Thymeleaf

4.2.2 application.yml — Cookie Segura

```
1 server.servlet.session:
2   cookie:
3     name: CARRITO_SESSION
4     http-only: true      # flag HttpOnly
5     secure: false       # true con HTTPS en produccion
6     same-site: strict   # flag SameSite = Strict
7     timeout: 30m
```

Listing 4.2: application.yml: Configuración de la cookie de sesión

4.2.3 model/Producto.java

```
1 package ec.edu.uteq.carrito.model;
2
3 import java.io.Serializable;
4
5 /**
6  * Modelo de producto.
7  * Implementa Serializable para persistencia en HttpSession.
8  */
9 public record Producto(String nombre, double precio)
10    implements Serializable {
11     private static final long serialVersionUID = 1L;
12 }
```

Listing 4.3: Producto.java: Record implementando Serializable

4.2.4 controller/CarritoController.java

```
1 @Controller
2 @RequestMapping("/carrito")
3 public class CarritoController {
4
5     private static final String CLAVE_CARRITO = "carrito";
6
7     private static final Map<String, Double> PRECIOS = Map.of(
8         "Laptop", 450.0, "Mouse", 25.0,
9         "Teclado", 45.0, "Monitor", 180.0
10    );
11
12     @SuppressWarnings("unchecked")
13     private List<Producto> obtenerCarrito(HttpSession session)
14     {
15         List<Producto> carrito =
```

```

15         (List<Producto>) session.getAttribute (CLAVE_CARRITO
16         );
17         if (carrito == null) {
18             carrito = new ArrayList<>();
19             session.setAttribute (CLAVE_CARRITO, carrito);
20         }
21
22         return carrito;
23     }
24
25     @GetMapping
26     public String index (HttpSession session, Model model) {
27         List<Producto> carrito = obtenerCarrito (session);
28         double total = carrito.stream ()
29             .mapToDouble (Producto::precio).
30             sum ();
31         model.addAttribute ("carrito", carrito);
32         model.addAttribute ("catalogo", PRECIOS.keySet ());
33         model.addAttribute ("total", total);
34         model.addAttribute ("sessionId", session.getId ());
35         return "carrito/index";
36     }
37
38     @PostMapping ("/agregar")
39     public String agregar (@RequestParam String nombre,
40                           HttpSession session) {
41         Double precio = PRECIOS.get (nombre);
42         if (precio != null)
43             obtenerCarrito (session).add (new Producto (nombre,
44                                                         precio));
45         return "redirect:/carrito"; // PRG
46     }
47
48     @PostMapping ("/eliminar")
49     public String eliminar (@RequestParam int indice,
50                           HttpSession session) {
51         List<Producto> carrito = obtenerCarrito (session);
52         if (indice >= 0 && indice < carrito.size ())
53             carrito.remove (indice);
54         return "redirect:/carrito";
55     }
56
57     @PostMapping ("/limpiar")
58     public String limpiar (HttpSession session) {
59         session.removeAttribute (CLAVE_CARRITO);
60         return "redirect:/carrito";
61     }
62 }

```

Listing 4.4: CarritoController.java: Controlador con HttpSession

Capítulo 5

Análisis Comparativo

5.1 Comparación entre Plataformas

Cuadro 5.1: Comparación de implementación de sesiones entre las tres plataformas

Aspecto	PHP 8	ASP.NET Core 8	Spring Boot 3
API de sesión	\$_SESSION	ISession	HttpSession
Serialización	Automática (PHP)	Manual (JSON)	Automática (Serializable)
Configuración cookie	ini_set()	AddSession()	application.yml
Motor de vistas	PHP nativo	Razor (.cshtml)	Thymeleaf (.html)
Patrón PRG	header('Location:R)redirectToAction		redirect:/ruta
Eliminación de item	array_splice	RemoveAt(i)	remove(i)
Destruir sesión	session_destroy()	Session.Clear()	session.invalidate()

5.2 Resumen de Flags de Seguridad

Cuadro 5.2: Configuración de flags de seguridad por plataforma

Flag	PHP 8	ASP.NET Core 8	Spring Boot 3
Cookie Name	PHPSESSID	.AspNetCore.Session	CARRITO_SESSION
HttpOnly	ini_set(..., '1')	Cookie.HttpOnly = true	http-only: true
Secure	ini_set(..., '1')	SecurePolicy.Always	secure: true
SameSite	ini_set(..., 'Strict')	SameSiteMode.Strict	same-site: strict

Capítulo 6

Conclusiones

1. Las tres plataformas implementan el concepto de sesión de manera similar, variando únicamente en la API y la sintaxis propia de cada ecosistema.
2. El patrón **PRG** es esencial en aplicaciones con formularios para evitar el reenvío accidental de datos al recargar la página.
3. La validación del lado del servidor es indispensable: nunca se debe confiar en los datos enviados por el cliente, especialmente en precios o identificadores de producto.
4. La configuración de las banderas HttpOnly, Secure y SameSite=Strict en la cookie de sesión es una medida de seguridad básica que protege contra ataques XSS y CSRF.
5. En **ASP.NET Core**, los objetos complejos deben serializarse a JSON manualmente antes de almacenarse en ISession, ya que ésta no maneja serialización automática de objetos.
6. En **Spring Boot**, el modelo debe implementar Serializable para que pueda ser gestionado correctamente por el contenedor de sesiones de Jakarta EE.